

De la idea al Pull Request:

trabajo colaborativo con Git, GitHub y Gitflow

Guía paso a paso para un equipo de tres desarrolladores que construyen una misma aplicación en paralelo, se revisan el código entre sí y fusionan sus cambios siguiendo el modelo de ramas Gitflow.

Modalidad: equipo de exactamente 3 personas · **Duración:** 3 a 4 horas · **Nivel:** introductorio–intermedio · **Entregable:** repositorio en GitHub con historial de ramas y al menos tres Pull Requests fusionados.

1. Objetivo del taller

Al terminar, el equipo será capaz de:

- Configurar un repositorio remoto compartido y agregar colaboradores.
- Aplicar Gitflow simplificado: ramas `main`, `develop` y `feature/*`, más una rama `release/*`.
- Trabajar tres funcionalidades en paralelo sin pisarse el trabajo.
- Abrir, revisar, comentar, aprobar y fusionar Pull Requests.
- Detectar y resolver un conflicto de fusión.
- Publicar una versión etiquetada (`v1.0.0`) en `main`.

2. Roles del equipo

Repártanse estos tres roles. Son de coordinación: **las tres personas programan y abren Pull Requests.**

ROL	RESPONSABILIDAD PRINCIPAL	FUNCIONALIDAD QUE CONSTRUYE
Dev A Líder técnico	Crea el repositorio, la rama <code>develop</code> y la rama <code>release/*</code> . Vela por que se siga Gitflow.	<code>feature/formulario-reserva</code>
Dev B Colaborador	Revisa al menos un Pull Request y participa en la resolución del conflicto.	<code>feature/listado-horarios</code>
Dev C Colaborador	Revisa al menos un Pull Request y redacta el reporte de cierre.	<code>feature/autenticacion-usuarios</code>

3. Requisitos previos

- Git instalado en los tres computadores (`git --version` debe responder).
- Una cuenta de GitHub por persona.
- Un editor de código (por ejemplo, VS Code).
- Haber definido el proyecto en el Tema 3 y el backlog en el Tema 4. Se trabajará sobre el *Sistema de reserva de laboratorios*.

4. Proyecto base

No se necesita backend. Cada funcionalidad puede ser una página HTML/CSS estática dentro de una carpeta:

```
reserva-laboratorios/  
├─ README.md  
├─ .gitignore  
├─ index.html  
└─ src/  
    ├─ formulario-reserva/      (Dev A)  
    ├─ listado-horarios/       (Dev B)  
    └─ autenticacion/          (Dev C)
```

Que cada persona trabaje dentro de su propia carpeta reduce (pero no elimina) los conflictos: el `README.md` será el archivo que todos toquen, y ahí se practicará el conflicto a propósito.

5. Recorrido paso a paso

Paso 1 — Preparación del repositorio Dev A

1. En GitHub: *New repository* → nombre `reserva-laboratorios` → visibilidad **Private** → marcar "Add a README file", elegir licencia MIT y plantilla de `.gitignore` (por ejemplo, *Node*).
2. *Settings* → *Collaborators* → *Add people*: invitar a Dev B y Dev C con sus usuarios de GitHub. Ellos deben aceptar la invitación por correo.
3. Dev A configura su identidad (una sola vez por computador) y clona:

```
git config --global user.name "Nombre de Dev A"
git config --global user.email "deva@correo.com"
git clone https://github.com/dev-a/reserva-laboratorios.git
cd reserva-laboratorios
```

Paso 2 — Crear y publicar la rama `develop` Dev A

```
git checkout -b develop
git push -u origin develop
```

En GitHub, *Settings* → *Branches*: marcar `develop` como rama por defecto (opcional pero recomendado) para que los Pull Requests apunten a ella automáticamente.

Paso 3 — Dev B y Dev C clonan y se ubican en `develop` Dev B Dev C

```
git config --global user.name "Nombre"
git config --global user.email "correo@correo.com"
git clone https://github.com/dev-a/reserva-laboratorios.git
cd reserva-laboratorios
git checkout develop
```

Paso 4 — Cada quien crea su rama `feature/*` A B C

Siempre partiendo de `develop` actualizado:

```
git checkout develop
git pull origin develop
git checkout -b feature/formulario-reserva # Dev A
# git checkout -b feature/listado-horarios # Dev B
# git checkout -b feature/autenticacion-usuarios # Dev C
```

Paso 5 — Trabajo individual con commits atómicos (A) (B) (C)

Cada persona construye su parte y guarda avances en commits pequeños y con mensajes claros (en imperativo, describiendo el qué y el porqué):

```
git add src/formulario-reserva/  
git commit -m "Agrega formulario de reserva con selección de laboratorio y fecha"  
  
git add src/formulario-reserva/estilos.css  
git commit -m "Aplica estilos base al formulario de reserva"
```

Regla del equipo: ningún commit con mensaje "cambios", "arreglos" o "wip". Cada mensaje debe poder leerse en el historial y entenderse sin abrir el código.

Paso 6 — Mantenerse sincronizado con `develop` (A) (B) (C)

Al menos una vez por sesión, traer al día la rama propia con lo que el equipo ya integró:

```
git fetch origin  
git merge origin/develop  
# resolver conflictos si aparecen, luego continuar
```

Integrar seguido = conflictos pequeños. Integrar solo al final = conflictos enormes.

Paso 7 — Publicar la rama y abrir el Pull Request (A) (B) (C)

```
git push -u origin feature/formulario-reserva
```

1. GitHub muestra un aviso "Compare & pull request" → clic.
2. Base: `develop` · Compare: `feature/formulario-reserva`. **Nunca hacia `main`**.
3. Título claro. En la descripción: qué se hizo, por qué, cómo probarlo, y "Closes #N" si hay un issue asociado.
4. *Reviewers*: asignar a las otras dos personas del equipo.

Paso 8 — Revisión de código cruzada (B) revisa a (A), (C) revisa a (B), (A) revisa a (C)

- Abrir la pestaña *Files changed* del Pull Request.
- Dejar mínimo **dos comentarios**: una sugerencia concreta y una pregunta.
- Elegir *Approve* si está bien, o *Request changes* si algo debe corregirse.
- Quien recibe comentarios responde cada uno y, si aplica, corrige y vuelve a hacer `git push` a la misma rama: el Pull Request se actualiza solo.

Paso 9 — Conflicto de fusión, provocado y resuelto A y B

Dev A y Dev B editan **la misma línea** del `README.md` (por ejemplo, la descripción del proyecto), cada uno en su rama, y ambos fusionan a `develop`. El segundo en fusionar verá el conflicto:

```
git checkout develop
git pull origin develop
git merge feature/listado-horarios
# CONFLICT (content): Merge conflict in README.md
```

Git marca el archivo así:

```
<<<<<< HEAD
Descripción escrita por Dev A
=====
Descripción escrita por Dev B
>>>>>> feature/listado-horarios
```

Se edita el archivo a mano, se deja el texto final acordado (sin las marcas `<<<` `===` `>>>`), y se cierra el conflicto:

```
git add README.md
git commit -m "Resuelve conflicto de fusion en README (descripcion unificada)"
git push origin develop
```

Paso 10 — Fusionar los tres Pull Requests A B C

- Con al menos una aprobación por PR, usar *Merge pull request* en GitHub.
- Marcar *Delete branch*: la rama `feature/*` ya cumplió su función.
- Todo el equipo actualiza su copia local:

```
git checkout develop
git pull origin develop
git branch -d feature/mi-rama-ya-fusionada
```

Paso 11 — Preparar la versión con `release/*` Dev A

```
git checkout develop
git pull origin develop
git checkout -b release/1.0.0
# ajustes finales: numero de version en README, CHANGELOG, revision general
git add .
git commit -m "Prepara version 1.0.0"
git push -u origin release/1.0.0
# En GitHub: Pull Request  release/1.0.0 -> main
```

Paso 12 — Publicar en `main` y etiquetar Dev A

```
git checkout main
git pull origin main
git tag -a v1.0.0 -m "Primera version del sistema de reserva de laboratorios"
git push origin v1.0.0
# sincronizar la version de vuelta a develop
git checkout develop
git merge main
git push origin develop
```

Paso 13 — Reporte de cierre Dev C

Respondan como equipo, en un archivo `CIERRE.md` dentro del repositorio:

1. ¿Cuántos Pull Requests abrieron y cuántos commits tiene `develop`?
2. ¿Qué conflicto tuvieron y cómo lo resolvieron?
3. ¿Cuál comentario de revisión fue el más útil y por qué?
4. ¿Qué harían distinto la próxima vez para reducir conflictos?

6. Chuleta de Gitflow

RAMA	NACE DE	SE FUSIONA EN	PARA QUÉ
<code>main</code>	—	—	Código en producción, siempre estable.
<code>develop</code>	<code>main</code>	<code>main</code> vía <code>release/*</code>	Integración de funcionalidades terminadas.
<code>feature/*</code>	<code>develop</code>	<code>develop</code> vía Pull Request	Una funcionalidad aislada.
<code>release/*</code>	<code>develop</code>	<code>main</code> y <code>develop</code>	Últimos ajustes antes de publicar una versión.
<code>hotfix/*</code>	<code>main</code>	<code>main</code> y <code>develop</code>	Corrección urgente en producción.

7. Errores frecuentes y cómo evitarlos

SÍNTOMA	CAUSA HABITUAL	SOLUCIÓN
"Se borró mi trabajo al fusionar"	Se resolvió mal un conflicto y se descartó un lado.	Revisar el conflicto con calma; usar <code>git log</code> y <code>git revert</code> , nunca <code>reset --hard</code> sin entenderlo.
El Pull Request muestra cientos de archivos cambiados	La rama <code>feature/*</code> se creó desde un <code>develop</code> desactualizado.	Hacer <code>git merge origin/develop</code> en la rama y volver a subir.
"No puedo hacer push"	Otra persona subió cambios a esa rama antes.	<code>git pull --rebase</code> y luego <code>git push</code> .
Commits con el autor equivocado	No se configuró <code>user.email</code> en ese computador.	Configurarlo antes del primer commit del taller.

8. Rúbrica de evaluación (100 puntos)

CRITERIO	PUNTOS
Repositorio con ramas <code>main</code> , <code>develop</code> y al menos 3 <code>feature/*</code> visibles en el historial	20
Al menos 3 Pull Requests hacia <code>develop</code> , cada uno con descripción y revisores asignados	20
Cada Pull Request tiene comentarios de revisión reales de otra persona (no solo "LGTM")	15
Un conflicto de fusión resuelto, visible en el historial de commits	15
Rama <code>release/1.0.0</code> fusionada a <code>main</code> y etiqueta <code>v1.0.0</code> publicada	15
Commits atómicos con mensajes descriptivos (no "wip", "cambios", "arreglos")	10
Archivo <code>CIERRE.md</code> con las cuatro respuestas del reporte de cierre	5

“Educar no es llenar sino encender.” — Jorge Wagensberg, profesor y físico español.

Profe Daniel Moraj · Medellín, 2026 · Curso Diseño de Aplicaciones — Material educativo de acceso abierto.